SESIÓN 5 - 6 DE NOVIEMBRE

# API REST con Spring Boot

Semana 3 · RAP 220501096-02 y 220501096-04 · Evidencias GA7-AA2-EV02, AA5-EV01 y AA5-EV03

# ¿Por qué Spring Boot?

## Java Tradicional




## El Enfoque Spring Boot




**Autoconfiguración:** Convención sobre configuración. Spring decide por ti para que arranques en minutos.

**Starters:** Paquetes listos (web, data-jpa) que traen el ecosistema de dependencias necesario de forma modular.




**Tomcat Embebido:** No necesitas configurar un servidor externo; la aplicación compila y corre por sí sola.

# ¿Qué es Spring Boot y sus Pilares Fundamentales?

## Autoconfiguración

- Detección automática de dependencias en el classpath.
- Configura Beans por defecto sin XML ni anotaciones manuales extensas.
- Reduce el tiempo de setup de proyectos Java a minutos.

## Tomcat Embebido

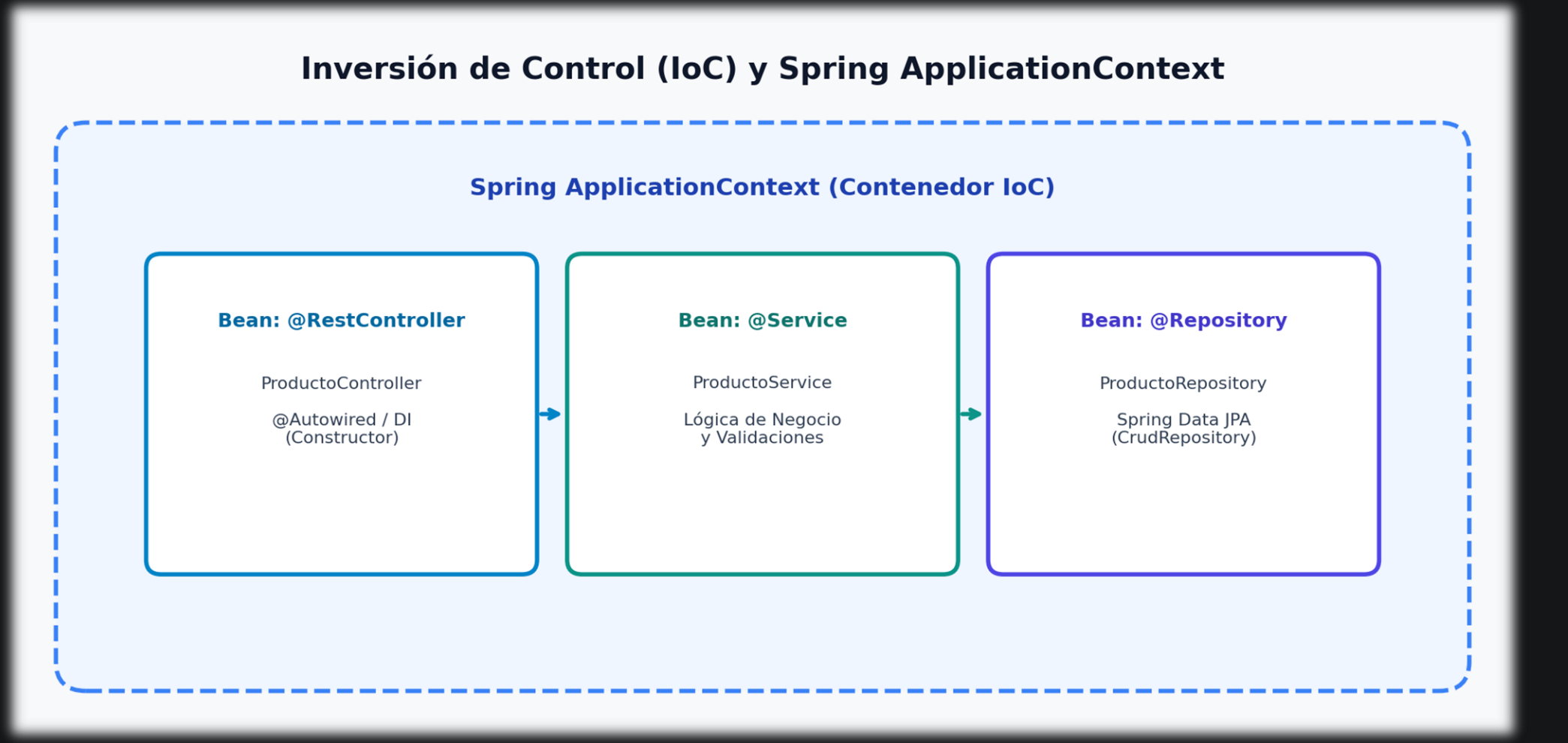
- Servidor Web Apache Tomcat integrado por defecto en puerto 8080.
- Elimina la necesidad de desplegar archivos WAR externos.
- Ejecución directa como aplicación independiente mediante `java -jar`.

## Starters Reutilizables

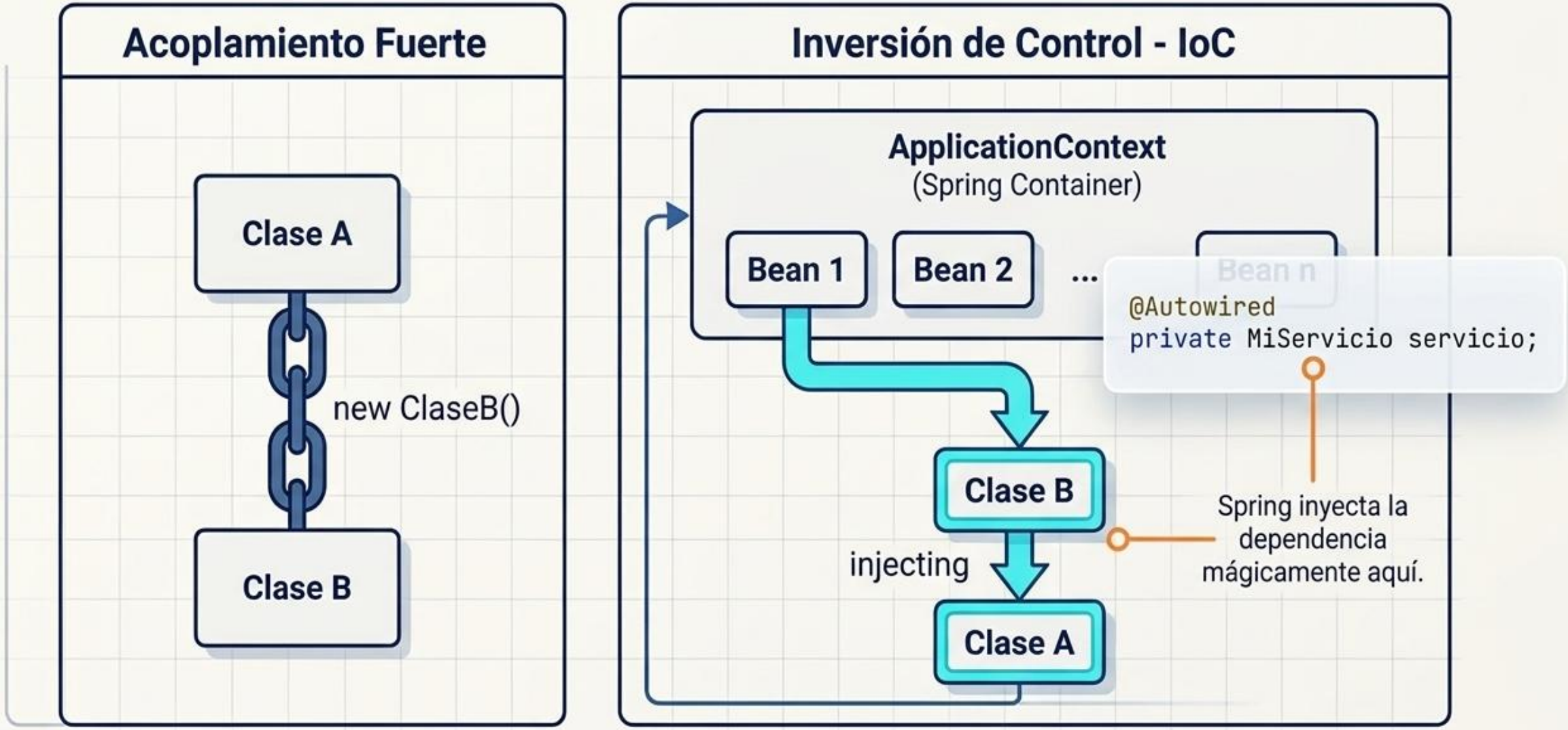
- Agrupaciones de librerías listas para usar.
- `spring-boot-starter-web` para APIs RESTful.
- `spring-boot-starter-data-jpa` para conexión y persistencia relacional SQL.

# Inversión de Control (IoC) y Spring ApplicationContext

El contenedor IoC de Spring administra el ciclo de vida y la instanciación de los Beans, eliminando el uso directo del operador `new` en las clases cliente.



# El Corazón del Sistema: Inversión de Control




Código espagueti. Las clases crean sus propias dependencias. Difícil de probar y mantener.

IoC traslada la responsabilidad de crear objetos al framework. Desacoplamiento total mediante Inyección de Dependencias.

# Inyección de Dependencias (DI) por Constructor

## Inyección por Constructor (Recomendado)

- Permite declarar atributos como `private final` garantizando inmutabilidad.
- Facilita la creación de pruebas unitarias al no requerir el contexto de Spring activo.
- Evita errores de referencias nulas (`NullPointerException`) en tiempo de ejecución.

## Código de Ejemplo Java

- @Service

```
public class ProductoService {

    private final ProductoRepository repo;

    // Inyección explícita por constructor
    public ProductoService(ProductoRepository r) {
        this.repo = r;
    }
}
```

# La Anotación `@SpringBootApplication`

## Componentes Integrados

- `@Configuration`: Define la clase como fuente de definiciones de Beans.
- `@EnableAutoConfiguration`: Habilita el escaneo de autoconfiguración de Spring Boot.
- `@ComponentScan`: Busca componentes anotados (`@Component`, `@Service`, `@Repository`) en el paquete actual y subpaquetes.

## Clase Principal

- @SpringBootApplication

```
public class KultApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            KultApplication.class, args);
    }
}
```

# Anotaciones Estereotipo de Componentes

## @RestController

- Especialización para la capa de presentación REST.
- Combina `@Controller` y `@ResponseBody`.
- Convierte automáticamente las respuestas Java a formato JSON.

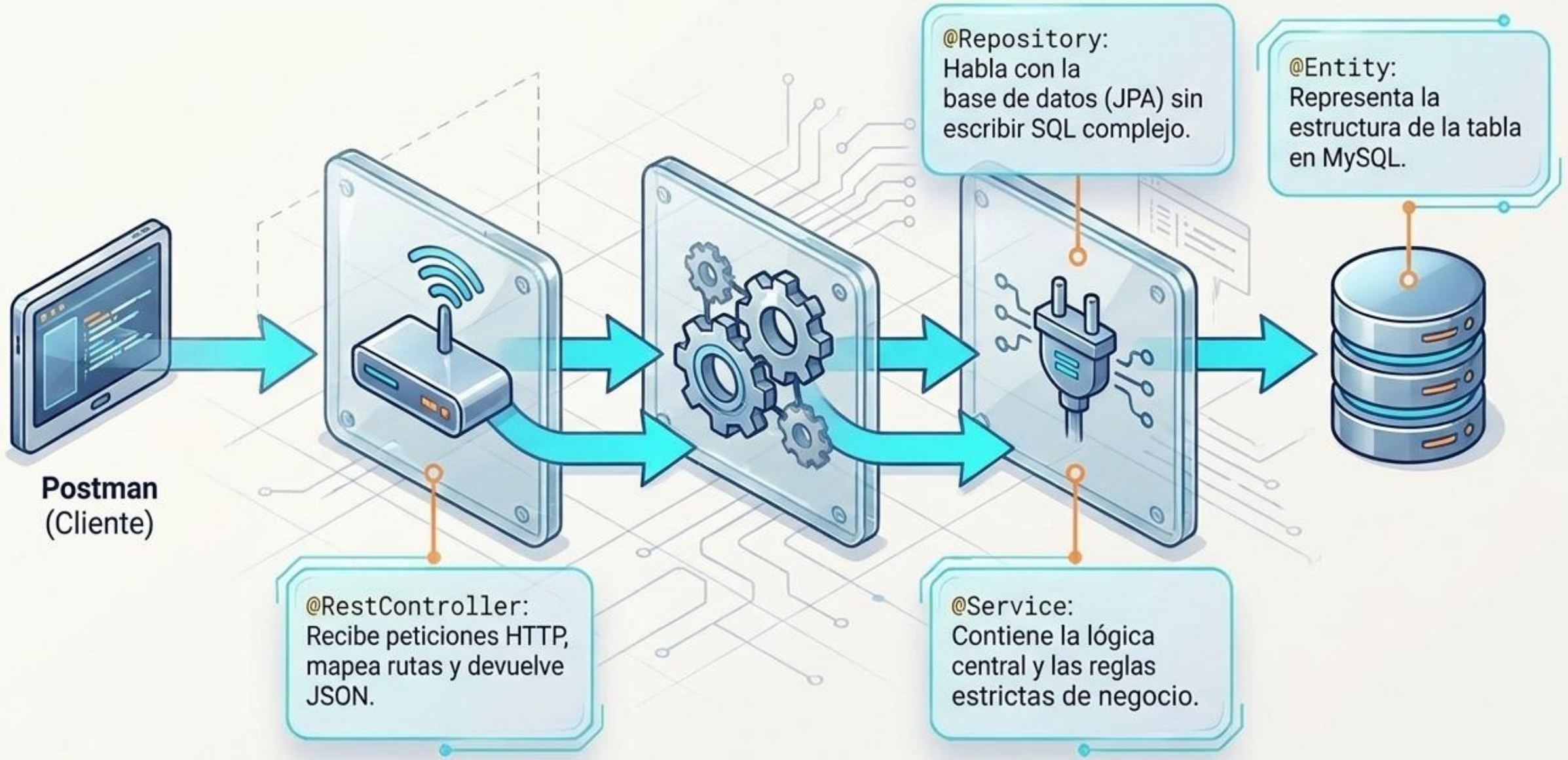
## @Service

- Contiene la lógica de negocio del sistema.
- Aplica reglas de validación, cálculos y procesamiento.
- Coordina operaciones transaccionales.

## @Repository

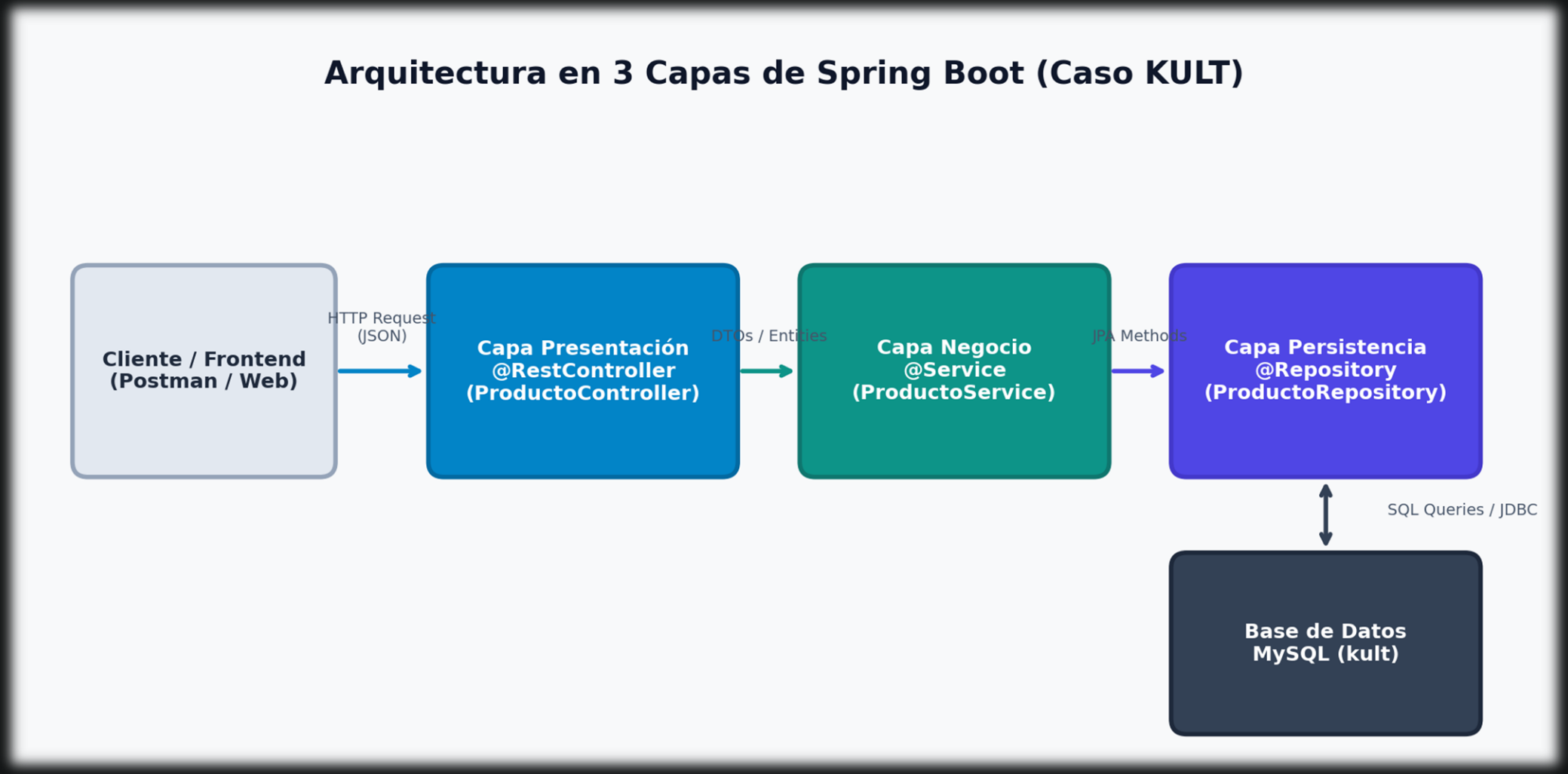
- Encargado del acceso y persistencia de datos.
- Traduce excepciones SQL relacionales a excepciones de Spring.
- Interactúa directamente con JPA / MySQL.

# La Arquitectura por Capas: El Mapa del Tesoro



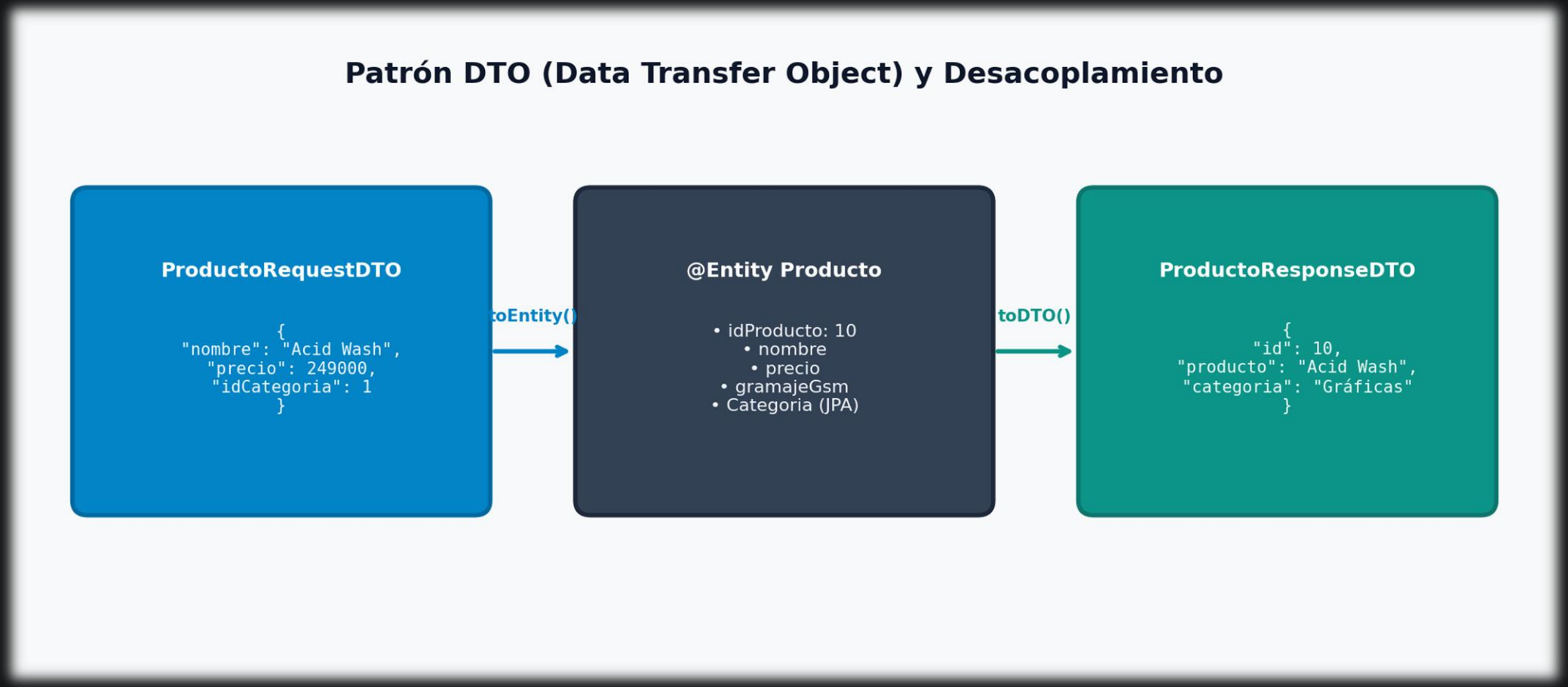
# Arquitectura en 3 Capas del Proyecto KULT

Organización modular con flujo unidireccional: Controller (endpoints) → Service (negocio) → Repository (persistencia MySQL).



# Patrón DTO (Data Transfer Object) y Aislamiento

Los DTOs ocultan campos sensibles de la base de datos, evitan ciclos de serialización JSON en relaciones JPA y desacoplan la API pública.



# Matriz de Responsabilidad Única (SRP) por Capa

## Controller

- PERMITIDO: Recibir HTTP, validar DTOs, invocar servicio y retornar ResponseEntity.
- PROHIBIDO: Escribir SQL, ejecutar lógica de inventario o acceder a DB directamente.

## Service

- PERMITIDO: Validar precios, calcular totales de compra, coordinar transacciones.
- PROHIBIDO: Leer parámetros HTTP o formatear respuestas JSON.

## Repository

- PERMITIDO: Ejecutar consultas SQL/JPQL y mapear filas a entidades JPA.
- PROHIBIDO: Aplicar reglas de negocio o lanzar excepciones HTTP.

# Inyección de Dependencias (DIP) con Interfaces

## Desacoplamiento mediante Abstracciones

- El servicio `ProductoService` no depende de una clase concreta MySQLDAO, sino de la interfaz `ProductoRepository`.
- Permite cambiar la base de datos o simular repositorios en pruebas unitarias (*Mocks*) sin modificar el código de negocio.
- Demostración de código:

```
public interface ProductoRepository extends JpaRepository<Producto, Integer> { }

@Service
public class ProductoService {
    private final ProductoRepository repo; // Depende de la interfaz
}
```

# Centralización de Errores con `@RestControllerAdvice`

## Ventajas de `@RestControllerAdvice`

- Captura excepciones no controladas lanzadas desde cualquier controlador.
- Retorna respuestas formateadas en JSON con código HTTP consistente (400, 404, 500).
- Evita bloques `try-catch` repetitivos en los controladores.

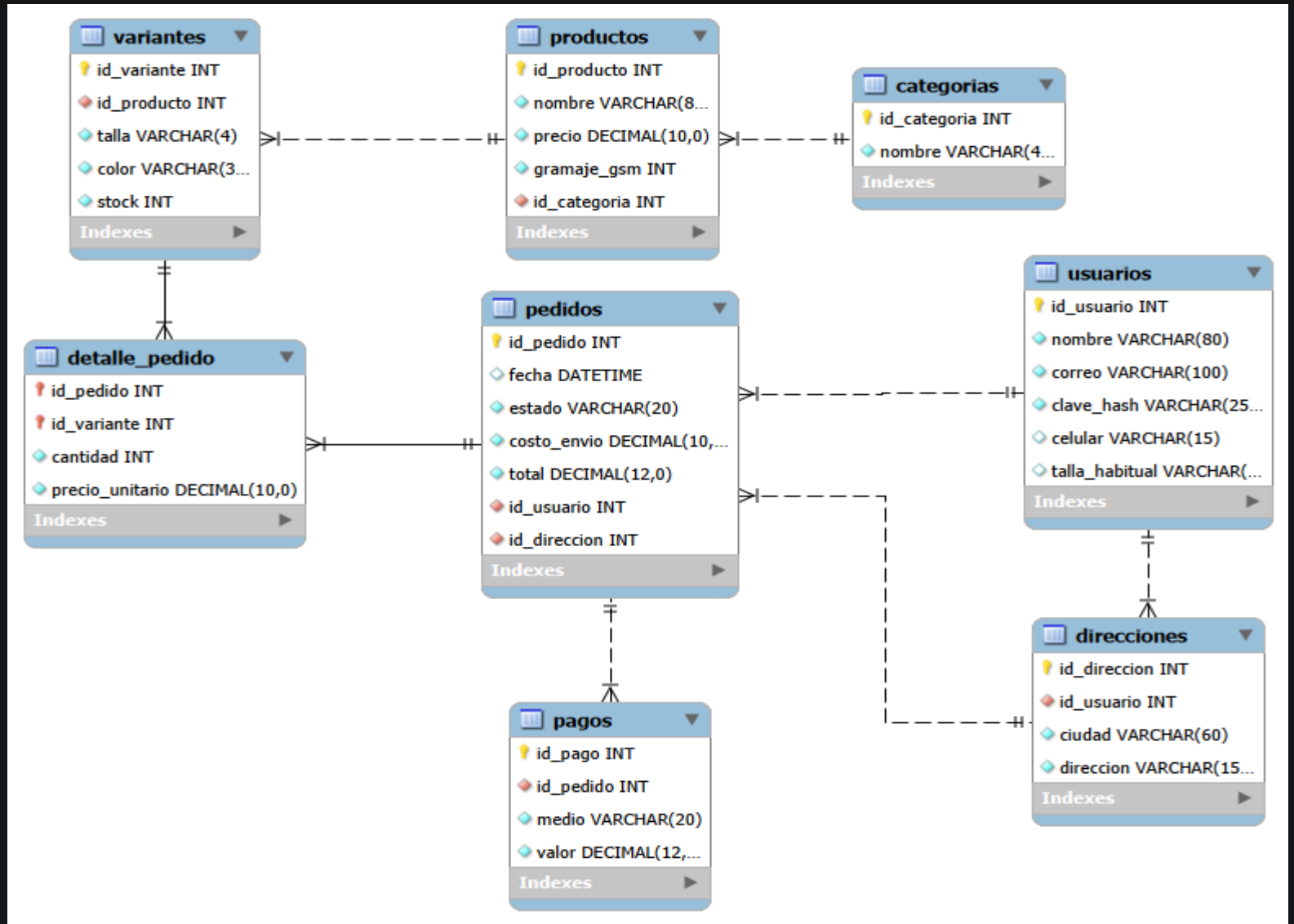
## Ejemplo de Interceptor

- @RestControllerAdvice

```
public class GlobalExceptionHandler {


@ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String>
handleNotFound(ResourceNotFoundException ex) {
        return Map.of("error", ex.getMessage());
    }
}
```

# Modelo Entidad-Relación de KULT (8 Tablas)



# Módulo de Catálogo: `categorias`, `productos` y `variantes`

### categorias

- id_categoria (PK, AUTO)
- nombre (VARCHAR 100, UNIQUE)
- descripcion (TEXT)
- Ejemplo: 'Camisetas Gráficas', 'Silueta Oversized'

### productos

- id_producto (PK, AUTO)
- nombre (VARCHAR 150)
- precio (DECIMAL 12,2)
- gramaje_gsm (INT, ej. 280, 350)
- id_categoria (FK)

### variantes

- id_variante (PK, AUTO)
- id_producto (FK)
- talla (VARCHAR 10, ej. 'L', 'M')
- color (VARCHAR 50)
- stock (INT NOT NULL DEFAULT 0)

# Módulo de Clientes: `usuarios` y `direcciones`

### Tabla `usuarios`

- id_usuario (PK AUTO)
- nombre, apellido (VARCHAR 100)
- correo (VARCHAR 150, UNIQUE)
- celular (VARCHAR 20)
- documento (VARCHAR 20, UNIQUE)

### Tabla `direcciones`

- id_direccion (PK AUTO)
- id_usuario (FK -> usuarios.id_usuario)
- direccion (VARCHAR 255)
- ciudad, departamento (VARCHAR 100)
- codigo_postal (VARCHAR 20)

# Módulo de Ventas: `pedidos`, `detalle_pedido` y `pagos`

## pedidos

- id_pedido (PK AUTO)
- fecha (DATETIME)
- estado (ENUM: PENDIENTE, PAGADO, ENVIADO...)
- costo_envio, subtotal, total
- id_usuario (FK), id_direccion (FK)

## detalle_pedido

- Clave Compuesta: (id_pedido PK/FK, id_variante PK/FK)
- cantidad (INT)
- precio_unitario (DECIMAL 12,2)

## pagos

- id_pago (PK AUTO)
- id_pedido (FK UNIQUE - Relación 1 a 1)
- medio ('TARJETAS CR/DB', 'PSE')
- valor, fecha_pago, estado

# Mapeo JPA: Entidad `@Entity Producto`

## Código de la Entidad Producto

```
• @Entity
@Table(name = "productos")
public class Producto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idProducto;

    @Column(nullable = false, length = 150)
    private String nombre;
    private BigDecimal precio;
    private Integer gramajeGsm;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "id_categoria")
    private Categoria categoria;
}
```

TEORÍA PARA PRINCIPIANTES · EJEMPLO BIBLIOTECA

# ¿Qué es una API REST?

```
GET /api/libros/1 HTTP/1.1
Host: localhost:8080

HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 1,
"titulo": "El principito",
"autor": "Saint-Exupéry" }
```

## API

Un contrato para que dos programas se hablen, como el mesero que lleva pedidos entre el cliente y la cocina.

## REST

Usa HTTP: el verbo dice la acción (GET leer, POST crear, PUT actualizar, DELETE borrar) y la ruta nombra el recurso.

## Respuesta

JSON más un código: 200 bien, 201 creado, 204 sin contenido, 400 datos inválidos, 404 no existe, 500 error del servidor.

TEORÍA PARA PRINCIPIANTES · EJEMPLO BIBLIOTECA

# Spring Boot y Maven

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

// En la terminal:
mvn clean package
java -jar target/biblioteca-0.0.1.jar
```

## Spring Boot

Framework de Java que arma el servidor y la configuración por ti, como una cocina ya equipada.

## Maven

Gestor de dependencias: en pom.xml listas las librerías y las descarga solo.

## Starters

Paquetes listos por tema: web para la API, data-jpa para la base de datos, mysql-connector para MySQL.

PASO 0 · PREPARACIÓN

# Qué debe estar instalado

```
java -version
mvn -version
mysql --version
git --version

// Spring Boot 3 exige Java 17 o superior
```

### JDK 17+ y Maven

Compilan y ejecutan el proyecto; el IDE puede ser IntelliJ IDEA o VS Code.

### MySQL con la base kult

Las tablas de la sesión 3 ya creadas con su script .sql.

### Postman y Git

Postman prueba los endpoints; Git guarda el trabajo por ramas.

CREAR EL PROYECTO

# Spring Initializr: start.spring.io

| Campo | Valor |
| --- | --- |
| Project y Language | Maven · Java |
| Spring Boot | 3.x estable (sin SNAPSHOT) |
| Group y Artifact | com.kult · tienda |
| Packaging y Java | Jar · 17 |
| Dependencies | Spring Web · Spring Data JPA · MySQL Driver |

Generate descarga un .zip: descomprímelo y ábrelo en el IDE. Lombok es opcional; en estas diapositivas los getters y setters se generan con el IDE.

# Fase 1: Generación del Proyecto (Spring Initializr)

## Configuración Base

- [x] **Gestor:** `Maven`
- [x] **Lenguaje:** `Java (JDK 17+)`
- [x] **Metadatos:** `Group: com.kult | Artifact: tienda`

## Dependencias Clave

### Spring Web
Para montar la API y los controladores REST.

### Spring Data JPA
Para el acceso a datos y mapeo objeto-relacional (ORM).

### MySQL Driver
Para la conexión nativa al motor de base de datos.

### Lombok
Magia para generar getters/setters automáticamente y limpiar código.

TEORÍA PARA PRINCIPIANTES · EJEMPLO BIBLIOTECA

# Arquitectura por capas

```
Cliente (Postman)
↓ HTTP
Controller ← recibe y responde
↓
Service ← reglas del negocio
↓
Repository ← consulta los datos
↓
Base de datos
```

## Analogía del restaurante

Controller es el mesero, Service el chef, Repository la despensa y la base de datos la bodega.

## Para qué sirve

Cada capa hace un solo trabajo, así es más fácil cambiar y probar el código.

## Regla

Cada capa solo habla con la siguiente; el controller no toca la base de datos.

ESTRUCTURA

# Carpetas y archivos del proyecto

```
tienda/
├─ pom.xml
├─ src/main/java/com/kult/tienda
│  ├─ TiendaApplication.java
│  ├─ model/Camisa.java
│  ├─ repository/CamisaRepository.java
│  ├─ service/CamisaService.java
│  └─ controller/CamisaController.java
└─ src/main/resources/
application.properties
```

### pom.xml

Lista las dependencias; Maven las descarga solo.

### TiendaApplication

@SpringBootApplication y main: arrancan el Tomcat embebido.

### Paquetes

Van dentro de com.kult.tienda para que Spring los detecte.

EJEMPLO BIBLIOTECA

# Configuración: application.properties

```
spring.datasource.url=
jdbc:mysql://localhost:3306/biblioteca
spring.datasource.username=root
spring.datasource.password=TU_CLAVE
server.port=8081
```

## Qué es

El archivo de ajustes de la aplicación: cambias valores sin tocar el código, como los ajustes del celular.

## La URL

jdbc:mysql://servidor:puerto/nombre_de_la_base, más usuario y clave.

## ddl-auto

none no hace nada, validate solo revisa, update modifica las tablas y create las recrea. La clave real no se sube a Git.

CONEXIÓN Y ARRANQUE

# application.properties y primera ejecución

```
spring.datasource.url=
jdbc:mysql://localhost:3306/kult
spring.datasource.username=root
spring.datasource.password=TU_CLAVE
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true

./mvnw spring-boot:run
```

### validate

Hibernate compara las entities con tus tablas; no crea ni borra nada.

### Arranque correcto

El log muestra "Tomcat started on port 8080" y "Started TiendaApplication".

### Errores comunes

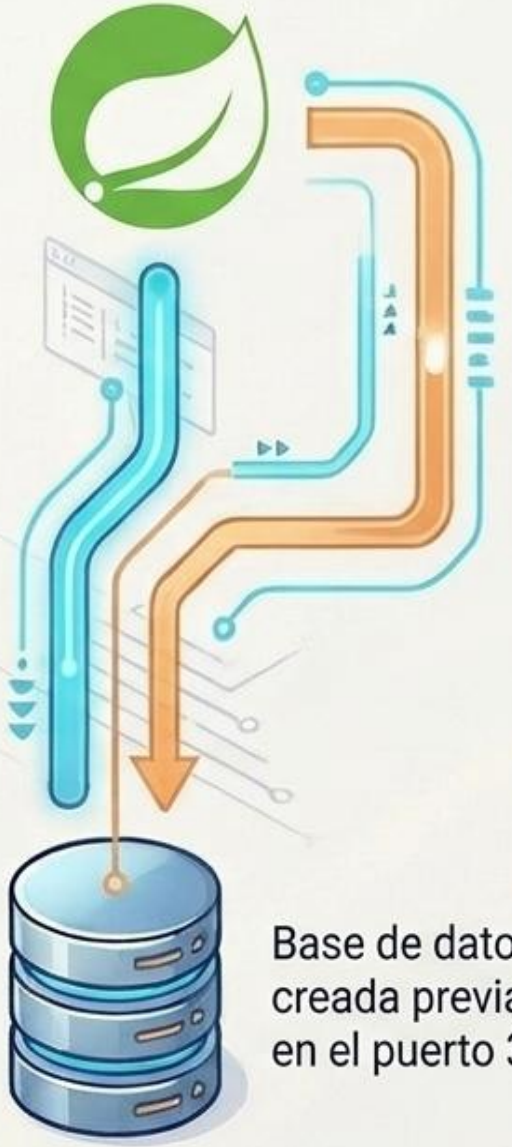
Access denied: usuario o clave. Unknown database: falta kult. Puerto ocupado: server.port=8081.

# Fase 2: Conexión a Base de Datos

## application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/kult
spring.datasource.username=root
spring.datasource.password=***
spring.jpa.hibernate.ddl-auto=update
```

**La magia de Hibernate/JPA:**
Escanea tus clases Java y crea o actualiza las tablas en MySQL automáticamente al arrancar. Sin scripts SQL manuales.



Base de datos 'kult' creada previamente en el puerto 3306.

EJEMPLO BIBLIOTECA

# Entity: una clase que es una tabla

```
@Entity
public class Libro {
@Id
@GeneratedValue(
strategy = GenerationType.IDENTITY)
private Integer id;
private String titulo;
private String autor;
// getters y setters
}
```

## Analogía del formulario

La clase es el formato en blanco y cada objeto es un formulario lleno, o sea una fila.

## Anotaciones

@Entity marca la tabla, @Id la clave primaria y @GeneratedValue deja que MySQL numere solo.

## Nombres

Libro es la tabla libro y anioPublicacion es la columna anio_publicacion; con @Table se cambia.

MODELO

# Entity Camisa

```
@Entity
@Table(name = "camisa")
public class Camisa {
@Id @GeneratedValue(
strategy = GenerationType.IDENTITY)
private Integer idCamisa;
private String nombre;
private BigDecimal precio;
private String tallas;
private Integer stock;
private Integer idCategoria;
// getters y setters
}
```

### Mapeo

idCamisa se mapea a id_camisa; cada objeto es una fila de la tabla.

### Getters y setters

Sin ellos el JSON sale vacío. Genéralos con el IDE.

### idCategoria

Entero simple por ahora; la relación @ManyToOne es el paso siguiente.

# Capa de Dominio: El Plano de la Base de Datos

**Java Class: Camisa.java**

```java
@Data
@Entity
@Table(name = "camisas")
public class Camisa {
    @Id
    @GeneratedValue(strategy =
     GenerationType.IDENTITY)
    private Long id;
    private String modelo;
    private Integer precio;
}
```

**@Entity** (Marca la clase como persistente")



**MySQL: camisas**

| | |
|---|---|
| 🔑 | **id** (PK, Auto-incremental) |
| | modelo (Varchar) |
| | precio (Int) |

**@Data** (Text: "Lombok inyecta getters/setters invisibles")

EJEMPLO BIBLIOTECA

# Repository: el acceso a los datos

```
public interface LibroRepository
extends JpaRepository<Libro, Integer> {
List<Libro> findByAutor(String autor);
}

// Uso:
repo.findAll();
repo.findById(1);
repo.save(libro);
repo.deleteById(1);
```

### Analogía del bibliotecario

No buscas en los estantes: le pides el libro y él lo trae.

### JpaRepository< Libro, Integer>

Indica la entity que maneja y el tipo de su id.

### Método derivado

findByAutor se convierte en SELECT con WHERE autor = ?, sin escribir SQL.

ACCESO A DATOS

# CamisaRepository

```
public interface CamisaRepository
extends JpaRepository<Camisa, Integer> {

List<Camisa> findByNombreContaining(
String texto);

List<Camisa> findByPrecioLessThan(
BigDecimal tope);
}
```

### JpaRepository

Ya trae save, findAll, findById y deleteById, sin escribir SQL.

### Métodos derivados

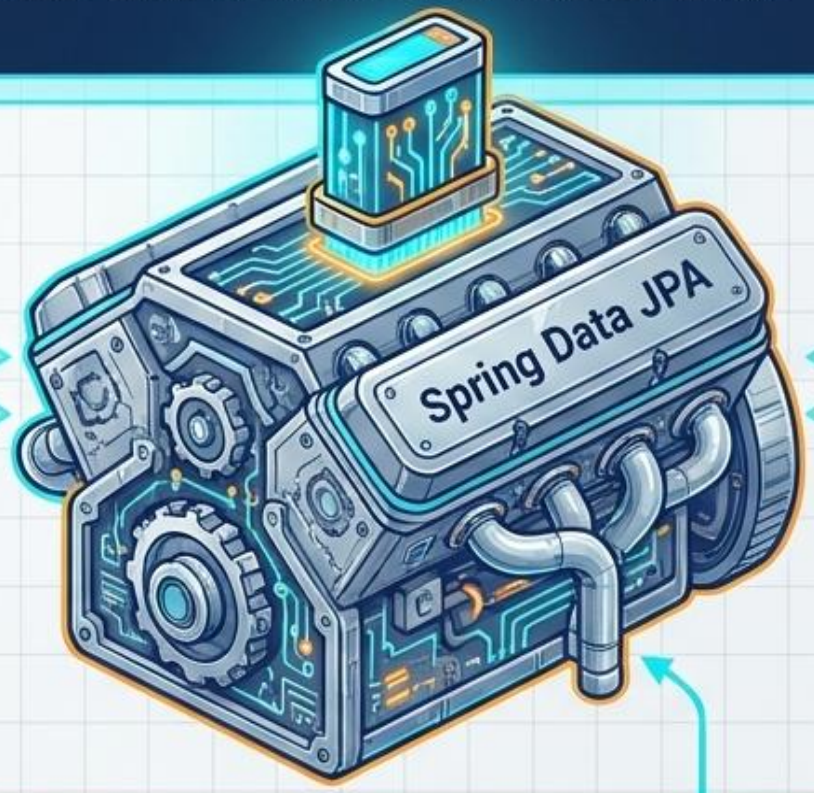
Spring genera la consulta a partir del nombre del método.

### Sin implementación

Solo se declara la interfaz; Spring crea el resto al arrancar.

# Capa de Acceso a Datos: El Poder de Spring Data JPA

```java
@Repository
public interface CamisaRepository extends JpaRepository<Camisa, Long> {
    // Interfaz vacía. La implementación la genera Spring.
}
```




save(Camisa)
-> Hace INSERT
o UPDATE

deleteById(Long)
-> Hace DELETE
WHERE id

findById(Long)
-> Hace SELECT
WHERE id

findAll()
-> Hace SELECT *

Al extender JpaRepository, heredas un arsenal completo de métodos CRUD sin escribir una sola línea de SQL.

# Capa de Negocio: Inyección de Dependencias en Acción

Desde el Controller (Petición)

```java
@Service
public class CamisaService {
    @Autowired // Inyección del contenedor IoC
    private CamisaRepository repository;

    public List<Camisa> obtenerTodas() {
        return repository.findAll();
    }
    public Camisa guardar(Camisa camisa) {
        // Validaciones de negocio aquí
        return repository.save(camisa);
    }
}
```

Hacia el Repository (Base de Datos)

El cerebro del sistema. @Service registra esta clase en el ApplicationContext. Nunca permitas que el Controlador hable directamente con la base de datos.

EJEMPLO BIBLIOTECA

# Service: las reglas del negocio

```
@Service
public class LibroService {
private final LibroRepository repo;
public LibroService(LibroRepository r) {
this.repo = r; }
public Libro crear(Libro l) {
if (l.getTitulo().isBlank())
throw new ResponseStatusException(
HttpStatus.BAD_REQUEST);
return repo.save(l); }
}
```

## Para qué sirve

Valida y decide antes de guardar, como el chef que revisa el pedido.

## Inyección de dependencias

Spring crea el repositorio y lo entrega por el constructor: no usas new.

## Errores

ResponseStatusException devuelve al cliente el código HTTP que elijas.

REGLAS DEL NEGOCIO

# CamisaService: listar y obtener

```
@Service
public class CamisaService {
private final CamisaRepository repo;
public CamisaService(CamisaRepository r) {
this.repo = r; }
public List<Camisa> listar() {
return repo.findAll(); }
public Camisa obtener(Integer id) {
return repo.findById(id).orElseThrow(() ->
new ResponseStatusException(
HttpStatus.NOT_FOUND)); }
```

### @Service

Marca la clase para que Spring la cree y la inyecte.

### Inyección

El repositorio llega por el constructor, sin new.

### 404

Si el id no existe, obtener lanza NOT_FOUND.

REGLAS DEL NEGOCIO

# CamisaService: crear, actualizar y eliminar

```
public Camisa crear(Camisa c) {
if (c.getPrecio().signum() <= 0)
throw new ResponseStatusException(
HttpStatus.BAD_REQUEST);
return repo.save(c); }
public Camisa actualizar(Integer id, Camisa c) {
Camisa e = obtener(id);
e.setPrecio(c.getPrecio());
e.setStock(c.getStock());
return repo.save(e); }
public void eliminar(Integer id) {
repo.delete(obtener(id)); }
}
```

### Validación

Un precio menor o igual a cero responde 400.

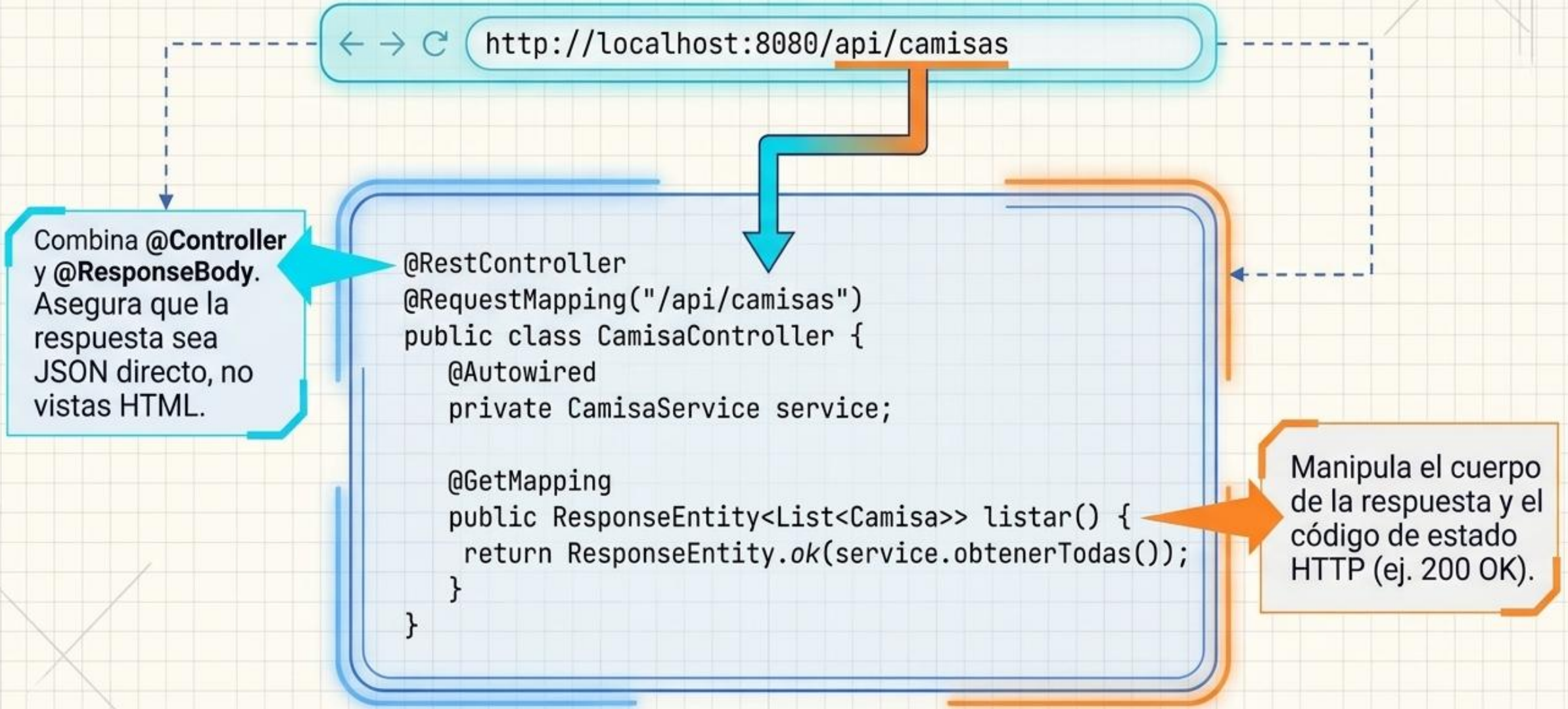
### Reutilizar obtener

Actualizar y eliminar devuelven 404 si el id no existe.

### Ampliar

Actualiza aquí también nombre y tallas si lo necesitas.

# Capa de Presentación: Mapeando la Web al Código



EJEMPLO BIBLIOTECA

# Controller: la puerta de la API

```
@RestController
@RequestMapping("/api/libros")
public class LibroController {
private final LibroService service;
public LibroController(LibroService s) {
this.service = s; }
@GetMapping("/{id}")
public Libro uno(@PathVariable Integer id) {
return service.obtener(id); }
}
```

## Analogía del mesero

Recibe la petición, la lleva a la cocina y devuelve la respuesta.

## Anotaciones por verbo

@GetMapping, @PostMapping, @PutMapping @DeleteMapping enlazan un verbo HTTP con método.

## Datos de entrada

@PathVariable toma el id de /api/libros/5, @RequestBody el JSON y @RequestParam el ?autor=Borges.

PASO 7 · ENDPOINTS

# CamisaController: consultas

```
@RestController
@RequestMapping("/api/camisas")
public class CamisaController {
private final CamisaService service;
public CamisaController(CamisaService s) {
this.service = s; }
@GetMapping
public List<Camisa> listar() {
return service.listar(); }
@GetMapping("/{id}")
public Camisa obtener(@PathVariable Integer id) {
return service.obtener(id); }
```

**@RestController**

Devuelve JSON en lugar de páginas HTML.

**@RequestMapping**

Ruta base de todos los métodos: /api/camisas.

**@PathVariable**

Toma el id de la URL, por ejemplo /api/camisas/1.

ENDPOINTS

# CamisaController: crear, actualizar y eliminar

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Camisa crear(@RequestBody Camisa c) {
return service.crear(c); }
@PutMapping("/{id}")
public Camisa actualizar(@PathVariable Integer id,
@RequestBody Camisa c) {
return service.actualizar(id, c); }
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void eliminar(@PathVariable Integer id) {
service.eliminar(id); }
}
```

### @RequestBody

Convierte el JSON enviado en un objeto Camisa.

### @ResponseStatus

POST responde 201 y DELETE responde 204.

### Reiniciar

Detén y vuelve a ejecutar la app antes de probar.

# Síntesis Analítica: Anatomía de un CRUD REST

| Operación | Verbo HTTP | Anotación Spring | Método JPA | Código HTTP |
|---|---|---|---|---|
| Crear | POST | `@PostMapping` | `.save()` | 201 Created |
| Leer | GET | `@GetMapping` | `.findAll()` / `.findById()` | 200 OK |
| Actualizar | PUT | `@PutMapping` | `.save()` (con ID) | 200 OK |
| Eliminar | DELETE | `@DeleteMapping` | `.deleteById()` | 204 No Content |

Anatomía de un endpoint REST. Cada operación de negocio se mapea estandarizadamente a un verbo HTTP y un estado de respuesta.

# Flujo End-to-End: El Viaje de un Payload




### 1. Payload POST

Entra JSON {modelo: Oversize, precio: 45} ⟶

### 2. Controller

Deserializa el JSON a Objeto Java Camisa.

### 3. Service

Valida los datos según reglas de negocio.

### 4. Repository

Hibernate genera INSERT INTO camisas...

### 5. MySQL

Fila guardada. Genera Auto-ID 1.

EJEMPLO BIBLIOTECA

# Postman: probar sin escribir un front

```
GET http://localhost:8080/api/libros

POST http://localhost:8080/api/libros
Body › raw › JSON:
{ "titulo": "El Aleph",
"autor": "Borges" }

// Esperado: 201 Created
```

### Analogía del control remoto

Envías órdenes a la API y ves cómo responde, sin construir una página.

### Partes de una petición

Método, URL, Headers (Content-Type: application/json) y Body.

### Leer la respuesta

Código de estado, tiempo y cuerpo JSON; en una Collection guardas las peticiones y usas {{baseUrl}}.

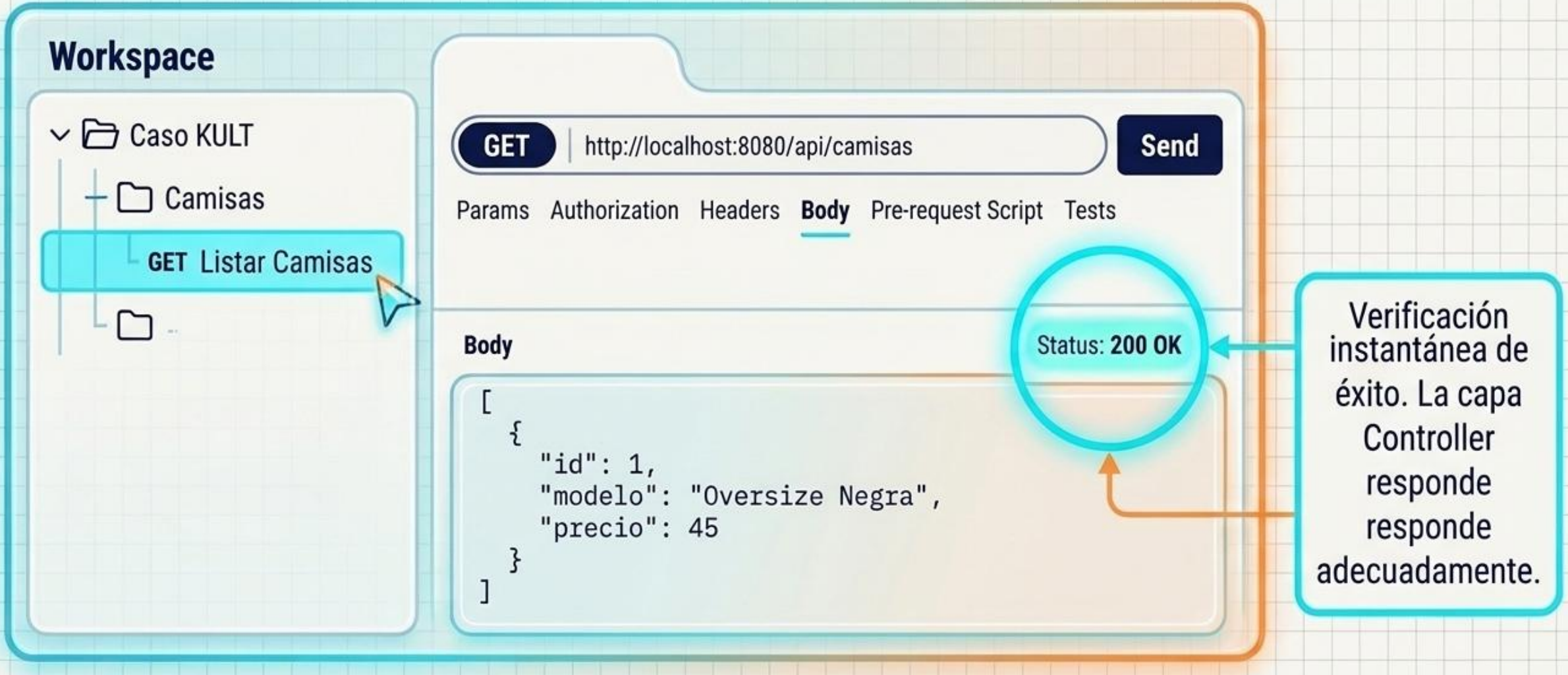
POSTMAN

# Colección de pruebas de la API

| Verbo | URL | Cuerpo | Esperado |
|---|---|---|---|
| GET | {{baseUrl}}/api/camisas | — | 200 y lista |
| POST | {{baseUrl}}/api/camisas | JSON | 201 |
| GET | {{baseUrl}}/api/camisas/{{id}} | — | 200 |
| PUT | {{baseUrl}}/api/camisas/{{id}} | JSON | 200 |
| DELETE | {{baseUrl}}/api/camisas/{{id}} | — | 204 |
| GET | {{baseUrl}}/api/camisas/999 | — | 404 |

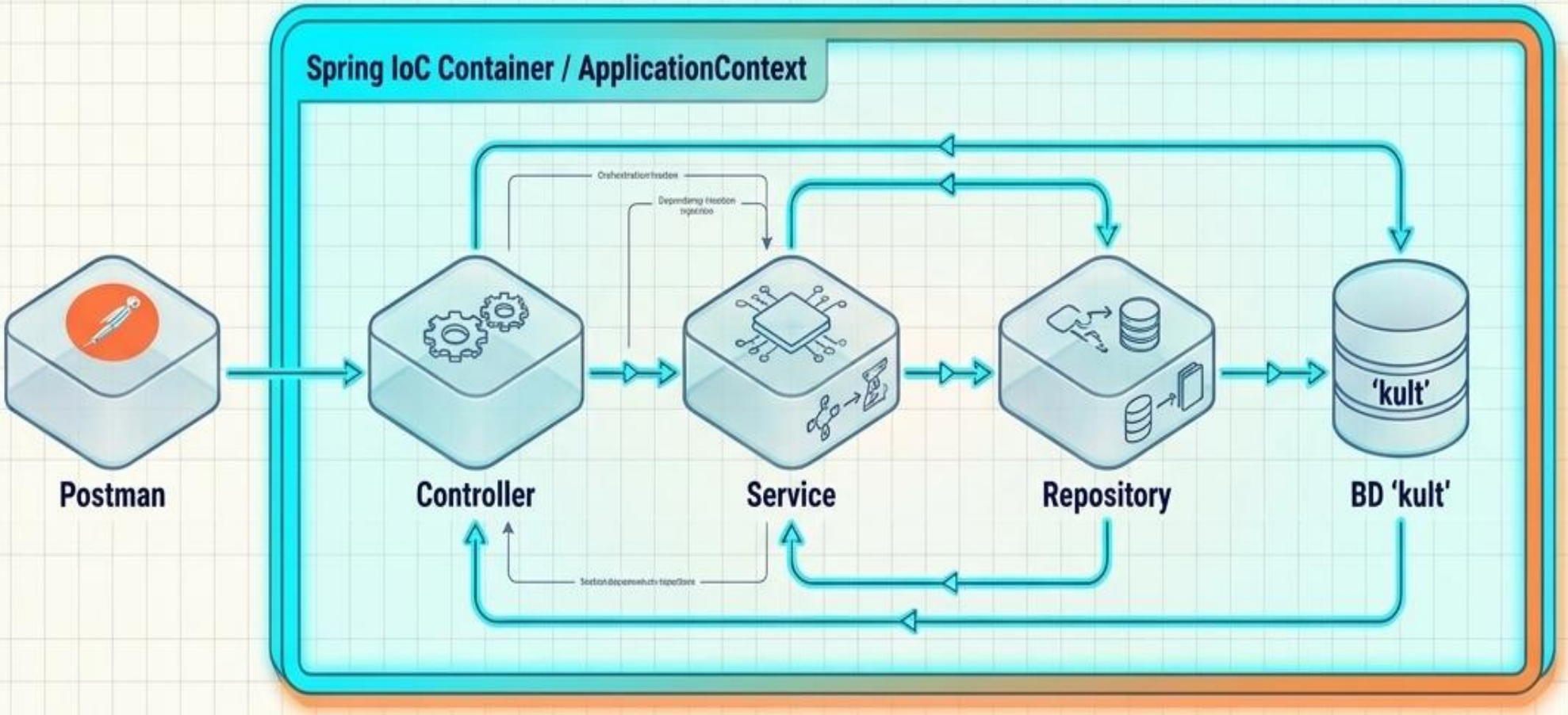
Variable baseUrl = http://localhost:8080 · POST en Body › raw › JSON: {"nombre":"System Crash Oversized Tee","precio":239000,"tallas":"S M L XL","stock":20,"idCategoria":2}

# Validación: Inspección Técnica con Postman




Organiza tus endpoints en colecciones lógicas para simular el comportamiento exacto de tu frontend.

# El Ecosistema KULT: Listo para Producción




- Has construido un sistema desacoplado, escalable y mantenible.
- Spring Boot gestiona la complejidad arquitectónica profunda (IoC, Autoconfiguración, Transacciones).
- Tu único trabajo: Concentrarte en la lógica de negocio que aporta valor al Caso KULT.

POSTMAN

# Pruebas automáticas en la pestaña Tests

```
pm.test("Status 201", () => {
pm.response.to.have.status(201);
});
pm.test("Devuelve idCamisa", () => {
const j = pm.response.json();
pm.expect(j.idCamisa).to.be.a("number");
pm.collectionVariables.set(
"id", j.idCamisa);
});
```

### Variable id

El POST guarda el id nuevo y las demás peticiones lo reutilizan.

### Orden del flujo

POST, GET lista, GET por id, PUT, DELETE y un GET final que debe dar 404.

### Collection Runner

Ejecuta toda la colección de una vez y muestra qué pruebas pasan.

PRÁCTICA Y ENTREGABLE

# API de camisas probada con Postman

### Pasos

Crear el proyecto · conectar a kult · Camisa y repositorio · servicio · controlador · colección Postman · un commit por paso en feature/api-camisas.

### Se valida si

GET devuelve las camisas en JSON; POST responde 201 y crea la fila en MySQL; DELETE responde 204 y un id inexistente 404; la colección pasa completa.

Entregable: repositorio con la API y la colección de Postman exportada (.json). No subas tu clave real de MySQL en application.properties.